Arquitectura de Software: Sistema Web de Gestión para una Agencia de Turismo

Rol: Arquitecto de Software Senior
Stack tecnológico: React + Node.js + Express + MySQL
Estilo arquitectónico recomendado: Arquitectura en capas con principios de Clean Architecture y separación por módulos.

1. Visión general de la arquitectura

Para este sistema recomiendo implementar una arquitectura cliente-servidor de tres capas, donde cada componente tenga responsabilidades independientes.

El objetivo es construir una aplicación que permita gestionar las operaciones de una agencia de turismo, como:

- Registro y administración de clientes.
- Gestión de paquetes turísticos.
- Administración de destinos.
- Reservaciones y cancelaciones.
- Gestión de pagos.
- Administración de usuarios y roles.
- Generación de reportes.

Diagrama general de arquitectura

CAPA DE PRESENTACIÓN

React.js

Componentes · Hooks · React Router · Axios

HTTPS / REST API / JSON

CAPA DE LÓGICA DE NEGOCIO

Node.js + Express

Rutas · Middleware · Controladores · Servicios

Autenticación JWT · Validaciones · Reglas de negocio

Consultas SQL mediante pool de conexiones

CAPA DE DATOS

MySQL

Usuarios · Clientes · Paquetes · Reservaciones · Pagos

¿Cómo funciona?

1. El usuario interactúa con la interfaz desarrollada en React.
2. React realiza peticiones HTTP al servidor Express.
3. Express recibe la solicitud y verifica autenticación, permisos y datos.
4. Los servicios ejecutan la lógica de negocio.
5. El repositorio correspondiente consulta o modifica MySQL.
6. El backend devuelve una respuesta JSON al frontend.

Esta separación permite modificar una capa sin afectar directamente las demás.

2. Estructura de carpetas sugerida

Recomiendo separar el proyecto en dos aplicaciones independientes: frontend y backend.

2.1. Frontend – React

frontend/
│
├── public/
│
├── src/
│   ├── assets/
│   │
│   ├── components/
│   │   ├── Navbar.jsx
│   │   ├── Sidebar.jsx
│   │   ├── Button.jsx
│   │   └── Modal.jsx
│   │
│   ├── pages/
│   │   ├── Login.jsx

```
│   │   ├── Dashboard.jsx
│   │   ├── Clientes.jsx
│   │   ├── Paquetes.jsx
│   │   ├── Reservaciones.jsx
│   │   └── Usuarios.jsx
│   │
│   ├── services/
│   │   ├── api.js
│   │   ├── authService.js
│   │   └── reservaService.js
│   │
│   ├── hooks/
│   │   └── useAuth.js
│   │
│   ├── context/
│   │   └── AuthContext.jsx
│   │
│   ├── routes/
│   │   └── ProtectedRoute.jsx
│   │
│   ├── layouts/
│   │   └── DashboardLayout.jsx
│   │
│   ├── App.jsx
│   └── main.jsx
│
└── package.json
```

Responsabilidad de cada directorio:

Directorio	Responsabilidad
components	Componentes reutilizables.
pages	Pantallas principales del sistema.
services	Comunicación con el backend.
hooks	Lógica reutilizable de React.
context	Estado global de autenticación.
routes	Protección y configuración de rutas.
layouts	Estructura visual de las páginas.

La ventaja principal es que los componentes no necesitan conocer cómo funciona internamente la base de datos.

2.2. Backend – Node.js + Express

Para el backend recomiendo una arquitectura modular organizada por responsabilidades.

```
backend/
│
├── src/
│   ├── config/
│   │   ├── database.js
│   │   └── environment.js
│   │
│   ├── modules/
│   │   ├── auth/
│   │   │   ├── auth.routes.js
│   │   │   ├── auth.controller.js
│   │   │   ├── auth.service.js
│   │   │   └── auth.repository.js
```

```
│   │   │
│   │   ├── usuarios/
│   │   │   ├── usuario.routes.js
│   │   │   ├── usuario.controller.js
│   │   │   ├── usuario.service.js
│   │   │   └── usuario.repository.js
│   │   │
│   │   ├── clientes/
│   │   ├── paquetes/
│   │   ├── reservaciones/
│   │   └── pagos/
│   │
│   ├── middlewares/
│   │   ├── auth.middleware.js
│   │   ├── role.middleware.js
│   │   ├── error.middleware.js
│   │   └── rateLimit.middleware.js
│   │
│   ├── utils/
│   │   ├── jwt.js
│   │   └── logger.js
│   │
│   ├── app.js
│   └── server.js
│
├── tests/
├── .env
```

├── package.json

└── .gitignore

¿Por qué esta estructura?

Cada módulo tiene sus propias rutas, controladores, servicios y repositorios.

Por ejemplo, si se necesita modificar el proceso de reservaciones, los cambios se realizan dentro de modules/reservaciones/, evitando afectar innecesariamente los demás módulos.

3. Patrones de diseño recomendados

Los patrones de diseño permiten organizar el código, reducir el acoplamiento y facilitar el mantenimiento.

Recomiendo implementar los siguientes:

3.1. Patrón MVC (Modelo - Vista - Controlador)

Se utiliza para separar las responsabilidades del sistema.

- Modelo: representa los datos y su acceso.
- Vista: corresponde a la interfaz desarrollada con React.
- Controlador: recibe las solicitudes y coordina las respuestas.

Ejemplo: cuando un administrador registra un nuevo paquete turístico, el controlador recibe la información y delega su procesamiento al servicio correspondiente.

3.2. Patrón Repository

Es especialmente importante para la conexión con MySQL.

Su función es separar las consultas SQL de la lógica de negocio.

Ejemplo:

```
// reserva.repository.js


const db = require("../../config/database");


const obtenerReservas = async () => {
  const [rows] = await db.execute(
    "SELECT * FROM reservaciones"
```

```
  );


  return rows;
};


module.exports = {
  obtenerReservas
};
```

El servicio utiliza el repositorio sin necesitar conocer cómo se ejecutan las consultas SQL.

3.3. Patrón Service Layer

Centraliza las reglas de negocio.

Por ejemplo, antes de registrar una reservación, el servicio puede verificar:

- Que el cliente exista.
- Que el paquete turístico esté disponible.
- Que existan cupos suficientes.
- Que la fecha de reservación sea válida.

Así se evita colocar toda la lógica dentro de los controladores.

3.4. Patrón Middleware

Permite ejecutar procesos antes de llegar a los controladores.

Algunos ejemplos:

- Verificar tokens JWT.
- Validar roles de usuario.
- Limitar solicitudes.
- Validar datos recibidos.
- Gestionar errores.

Recomendación: combinar estos patrones con principios SOLID, especialmente responsabilidad única y dependencia de abstracciones.

4. Seguridad: autenticación y protección de rutas

La seguridad debe implementarse tanto en el frontend como en el backend.

4.1. Autenticación mediante JWT

Se recomienda utilizar JSON Web Tokens (JWT) para gestionar las sesiones de los usuarios.

El funcionamiento sería el siguiente:

1. Inicio de sesión

El usuario introduce su correo y contraseña.

2. Validación de credenciales

Express consulta al usuario y compara la contraseña utilizando bcrypt o Argon2.

3. Generación del token

Si las credenciales son correctas, el servidor genera un JWT firmado con una clave secreta.

4. Almacenamiento seguro

El token de acceso se mantiene en memoria y se utiliza una cookie HttpOnly, Secure y SameSite para gestionar la sesión o el mecanismo de renovación.

5. Acceso a rutas protegidas

El middleware verifica el token antes de permitir el acceso a los recursos.

4.2. Ejemplo de generación de JWT

```
const jwt = require("jsonwebtoken");


const generarToken = (usuario) => {
  return jwt.sign(
    {
      id: usuario.id,
      rol: usuario.rol
    },
    process.env.JWT_SECRET,
    {
      expiresIn: "15m"
```

```
    }
  );
};

module.exports = generarToken;
```

La clave JWT_SECRET debe almacenarse en variables de entorno, nunca directamente en el código.

4.3. Protección de rutas

Se implementa un middleware que verifica el token antes de permitir el acceso.

```
const jwt = require("jsonwebtoken");

const verificarToken = (req, res, next) => {
  const token = req.cookies.accessToken;

  if (!token) {
    return res.status(401).json({
      mensaje: "Acceso no autorizado"
    });
  }

  try {
    req.usuario = jwt.verify(
      token,
      process.env.JWT_SECRET
    );

    next();
```

```
  } catch (error) {
    return res.status(403).json({
      mensaje: "Token inválido o expirado"
    });
  }
};

module.exports = verificarToken;
```

Para utilizar este ejemplo, se debe configurar previamente cookie-parser.

4.4. Control de roles (RBAC)

No todos los usuarios deben tener los mismos permisos.

Recomiendo establecer tres roles iniciales:

Rol	**Permisos**
Administrador	Acceso completo al sistema.
Empleado	Gestión de clientes y reservaciones.
Cliente	Consulta de paquetes y sus reservaciones.

Ejemplo de middleware:

```
const verificarRol = (...rolesPermitidos) => {
  return (req, res, next) => {

    if (!rolesPermitidos.includes(req.usuario.rol)) {
      return res.status(403).json({
        mensaje: "No tienes permisos"
      });
    }

    next();
```

```
  };
};
```

Uso:

```
router.delete(
  "/:id",
  verificarToken,
  verificarRol("administrador"),
  eliminarPaquete
);
```

Importante: React puede ocultar botones o páginas según el rol, pero la autorización real siempre debe verificarse en Express.

4.5. Protección de contraseñas

Nunca se deben guardar contraseñas en texto plano.

Recomiendo utilizar:

- Argon2id como primera opción para el almacenamiento de contraseñas.
- bcrypt como alternativa ampliamente utilizada.
- Validaciones de longitud y fortaleza.
- Protección contra intentos repetidos de inicio de sesión.
- HTTPS para cifrar la comunicación.

También es recomendable implementar recuperación de contraseñas mediante tokens temporales de un solo uso.

5. Escalabilidad: conexión eficiente con MySQL

Este es uno de los puntos más importantes de la arquitectura.

Un error frecuente consiste en crear una conexión nueva a MySQL cada vez que un usuario realiza una petición.

Si 1.000 usuarios realizan solicitudes simultáneamente, esto puede generar una cantidad excesiva de conexiones y provocar saturación del servidor.

5.1. Solución: Connection Pool

Se recomienda utilizar un pool de conexiones mediante el paquete mysql2.

El pool mantiene un conjunto limitado de conexiones reutilizables.

Servidor Express

Solicitudes simultáneas

Connection Pool

10–20

Conexiones iniciales ilustrativas

Conexión 1

Conexión 2

Conexión 3

Cada conexión disponible se reutiliza para diferentes consultas.

Si todas las conexiones están ocupadas, las solicitudes esperan a que alguna quede disponible, hasta que se alcance el tiempo límite configurado.

5.2. Configuración de MySQL

Archivo database.js:

```
const mysql = require("mysql2/promise");


const pool = mysql.createPool({


  host: process.env.DB_HOST,

  user: process.env.DB_USER,

  password: process.env.DB_PASSWORD,

  database: process.env.DB_NAME,


  waitForConnections: true,


  connectionLimit: 10,


  maxIdle: 10,
```

```
  idleTimeout: 60000,

  queueLimit: 100,

  enableKeepAlive: true

});

module.exports = pool;
```

Explicación de los parámetros:

Parámetro	**Función**
connectionLimit	Limita las conexiones simultáneas.
waitForConnections	Permite esperar cuando no hay conexiones disponibles.
maxIdle	Limita las conexiones inactivas.
idleTimeout	Controla cuánto tiempo permanecen inactivas.
queueLimit	Limita las solicitudes en espera.
enableKeepAlive	Ayuda a mantener conexiones activas.

Los valores son iniciales y deben ajustarse según la capacidad del servidor, el número de instancias del backend y la configuración de MySQL.

5.3. Buenas prácticas para evitar saturación

Además del pool, recomiendo:

1. Índices SQL: crear índices en columnas utilizadas frecuentemente para búsquedas, como id_cliente, id_paquete y fecha_reserva.
2. Paginación: evitar consultar miles de registros en una sola petición.
3. Consultas optimizadas: seleccionar únicamente las columnas necesarias.

4. Transacciones: utilizarlas cuando una operación involucre varias modificaciones relacionadas, como registrar una reservación y descontar cupos.
5. Caché: utilizar Redis para almacenar temporalmente información consultada frecuentemente.
6. Límites de solicitudes: implementar rate limiting para evitar abusos y sobrecarga.

5.4. Preparación para crecimiento

Cuando el sistema comience a recibir muchas peticiones, se puede evolucionar hacia una arquitectura horizontal:

Usuarios

Load Balancer

Distribución del tráfico

Backend 1

Express

Backend 2

Express

MySQL

Base de datos central

Redis: caché compartida entre instancias.

De esta manera, si aumenta el tráfico, se pueden agregar más instancias del backend sin tener que modificar toda la aplicación.

Es importante que el pool total de conexiones de todas las instancias no supere la capacidad configurada de MySQL.

6. Mantenibilidad: buenas prácticas de desarrollo

Para que el proyecto pueda crecer sin convertirse en un sistema difícil de modificar, recomiendo aplicar estas prácticas:

Práctica	**Beneficio**
SOLID	Reduce el acoplamiento entre componentes.
Clean Code	Facilita la lectura del código.

Separación por módulos	Organiza las funcionalidades.
Variables de entorno	Protege configuraciones sensibles.
Git y GitHub	Permite controlar versiones.
Pruebas unitarias	Detecta errores en funciones individuales.
Pruebas de integración	Comprueba la comunicación entre módulos.
Documentación API	Facilita el mantenimiento y la integración.

También recomiendo utilizar:

- Jest o Vitest: pruebas automatizadas.
- ESLint: análisis de calidad del código.
- Prettier: formato consistente.
- Swagger / OpenAPI: documentación de endpoints.
- Docker: entornos de desarrollo y despliegue reproducibles.

7. Resumen de tecnologías recomendadas

Componente	**Tecnología**	**Propósito**
Frontend	React	Interfaz de usuario.
Rutas frontend	React Router	Navegación y protección visual.
Peticiones HTTP	Axios	Comunicación con el backend.
Backend	Node.js + Express	API y lógica de negocio.
Autenticación	JWT	Gestión de sesiones.
Contraseñas	Argon2id	Hash seguro de contraseñas.
Base de datos	MySQL	Persistencia de información.
Conexiones	mysql2/promise	Pool de conexiones.
Caché	Redis	Optimización de consultas.
Seguridad	Helmet + Rate Limit	Protección del servidor.
Pruebas	Jest	Validación del funcionamiento.

Documentación Swagger	Documentación de la API.

8. Conclusión y recomendación arquitectónica

Para el sistema web de gestión de la agencia de turismo, la arquitectura más adecuada es una arquitectura modular en capas basada en principios de Clean Architecture.

Esta solución permite cumplir los tres atributos de calidad principales:

- Seguridad: mediante JWT, Argon2id, middleware de autenticación, control de roles y protección de rutas.
- Mantenibilidad: mediante separación de responsabilidades, patrones Repository y Service Layer, principios SOLID y módulos independientes.
- Escalabilidad: mediante Connection Pool, consultas optimizadas, Redis, balanceador de carga y múltiples instancias de Express.

Mi recomendación como arquitecto es comenzar con un monolito modular, no con microservicios. Para el tamaño inicial de una agencia turística, esta arquitectura ofrece una buena relación entre complejidad, rendimiento y facilidad de mantenimiento. Si el volumen de usuarios y operaciones crece considerablemente, se podrán separar determinados módulos en servicios independientes.

La decisión clave: construir desde el inicio un sistema modular, seguro y preparado para crecer, sin introducir complejidad innecesaria antes de necesitarla.